



Desarrollo de Aplicaciones Web con Blazor

2026

Profesores:

- Dr. Freddy Paz
- Dr. José Luis Corcuera
- Mag. Eric Huiza
- Mag. David Allasi

Introducción

¿Qué es Blazor?

- Blazor es un framework de interfaz de usuario web gratuito y de código abierto desarrollado por Microsoft.
- Permite a los desarrolladores crear aplicaciones web interactivas utilizando C# y HTML en lugar de JavaScript.
- Forma parte del ecosistema ASP.NET Core.
- Facilita el uso compartido de código y lógica de negocio entre el cliente y el servidor en el mismo lenguaje (C#).



Ventajas del Uso de Blazor

Desarrollo unificado: Permite utilizar C# en todo el espectro de la aplicación (frontend y backend).

Reutilización de código: Las bibliotecas de clases de .NET pueden ser compartidas directamente entre el servidor y la interfaz de cliente.

Ecosistema consolidado: Acceso directo a NuGet y a las amplias herramientas del entorno .NET.

Tipado estricto: C# proporciona comprobación en tiempo de compilación, lo que reduce errores en tiempo de ejecución frente a lenguajes interpretados.

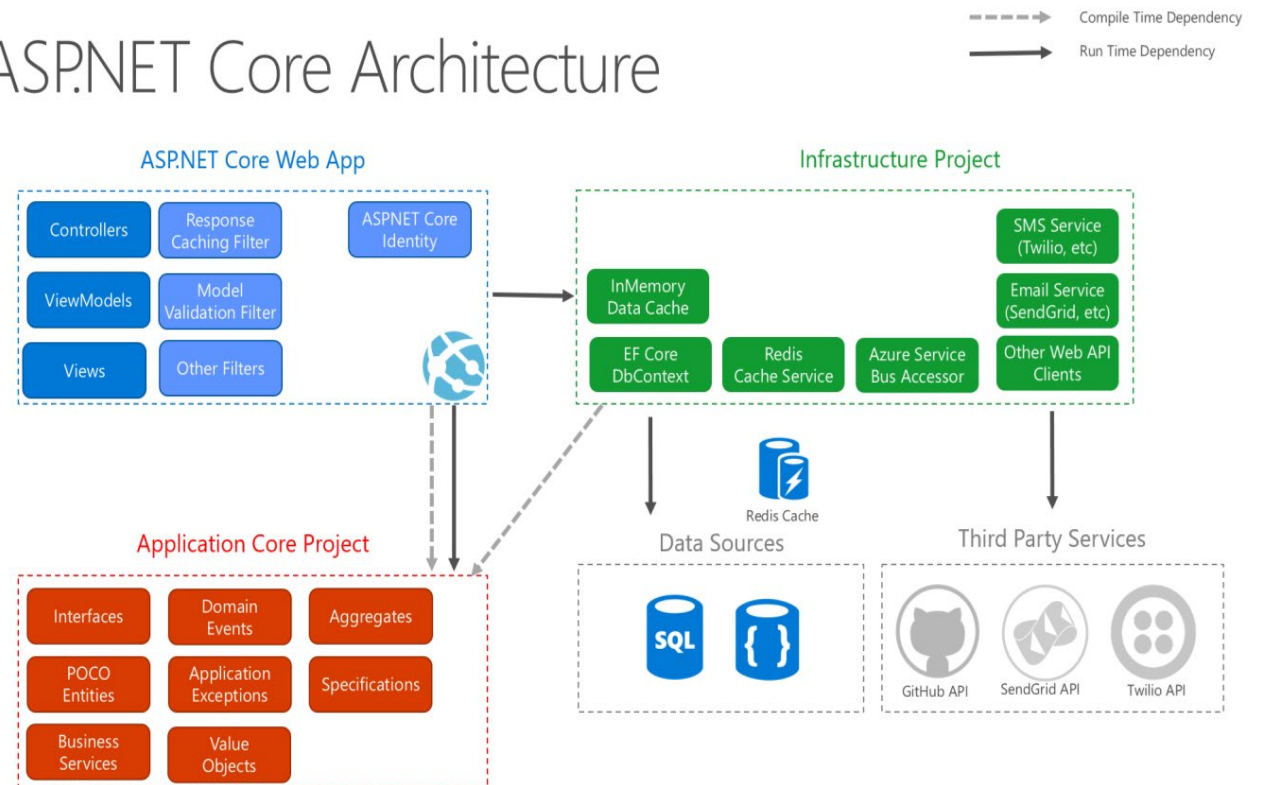


Fundamentos y Conceptos

Arquitectura Base de una Aplicación

1. La arquitectura de Blazor se basa en componentes.
2. Un componente es una unidad autónoma de interfaz de usuario, como un formulario, un cuadro de diálogo o una página entera.
3. Los componentes se renderizan en un árbol de renderizado, determinando qué elementos del DOM deben actualizarse.
4. La abstracción del renderizado permite que Blazor se ejecute tanto en WebAssembly como en el servidor sin cambiar la lógica del componente.

ASP.NET Core Architecture



Modelos de Renderizado

Blazor WebAssembly (WASM)

La aplicación, sus dependencias y el entorno de ejecución de .NET se descargan en el navegador. La ejecución es completamente del lado del cliente, permitiendo aplicaciones desconectadas y reduciendo la carga del servidor continuo.

Blazor Server

La aplicación se ejecuta en el servidor. Las actualizaciones de la interfaz de usuario, manejo de eventos y llamadas a JavaScript se gestionan mediante una conexión persistente (SignalR sobre WebSockets) con el cliente.

Consideraciones sobre el Modelo de Renderizado

.NET 8+ y Modos de Renderizado

A partir de .NET 8, Blazor introdujo el concepto de "Blazor Web App" que unifica los modelos. Permite definir el modo de renderizado (Server, WebAssembly o Auto) a nivel de componente individual, ofreciendo gran flexibilidad.

Consideraciones de Arquitectura

La selección del modelo afecta la seguridad, la latencia y el consumo de recursos. Server requiere conexión constante, mientras que WebAssembly implica un mayor tiempo de carga inicial pero operación autónoma posterior.

Estructura de un Proyecto Blazor

Directorios Principales

wwwroot: Contiene activos estáticos públicos (CSS, imágenes, JavaScript).

Components/Pages: Contiene los componentes enrutables (.razor) que actúan como páginas web.

Components/Layout: Contiene componentes de diseño principal (MainLayout.razor, NavMenu.razor).

Archivos Importantes

appsettings.json: Almacena configuraciones de la aplicación (cadenas de conexión, variables de entorno).

Program.cs: Punto de entrada de la aplicación; registra servicios y configura el pipeline HTTP.



El Archivo Program.cs

- Es el punto de entrada principal para cualquier aplicación ASP.NET Core.
- Aquí se inicializa el WebApplicationBuilder.
- Se realiza el registro de servicios, por ejemplo: `builder.Services.AddHttpClient()`.
- Se define el flujo de procesamiento HTTP.
- Se configuran los componentes interactivos de Blazor mediante llamadas como `AddInteractiveServerComponents()` o `AddInteractiveWebAssemblyComponents()`.

Archivos Importantes Adicionales

App.razor

Es el componente raíz de la aplicación. Contiene el componente Router que intercepta la navegación del navegador y renderiza la página que corresponde a la URL solicitada, definiendo el diseño principal (Layout) por defecto.

_Imports.razor

Contiene directivas Razor, principalmente declaraciones **@using**. Los espacios de nombres definidos en este archivo se aplican automáticamente a todos los componentes Razor dentro del mismo directorio y sus subdirectorios.



Creación de un Proyecto Blazor App

Seleccionar el Tipo de Aplicación

× ▼ Clear all

All languages ▼

All platforms ▼

All project types ▼



Blazor Web App

A project template for creating a Blazor web app that supports both server-side rendering and client-side user interfaces (UIs).

C#

Linux

macOS

Windows

Blazor

Cloud

Web

Configure la aplicación Blazor

Configure your new project

Blazor Web App C# Linux macOS

Project name

MiPrimeraBlazorApp

Location

E:\dev\2026-1\sesion-de-clase-13

Additional information

Blazor Web App C# Linux macOS Windows Blazor Cloud Web

Framework ⓘ

.NET 10.0 (Long Term Support)

Authentication type ⓘ

None

☒ Configure for HTTPS ⓘ

Interactive render mode ⓘ

Server

Interactivity location ⓘ

Global

☐ Include sample pages ⓘ

☐ Do not use top-level statements ⓘ

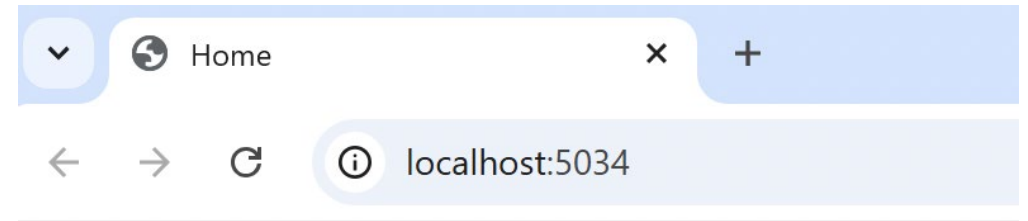
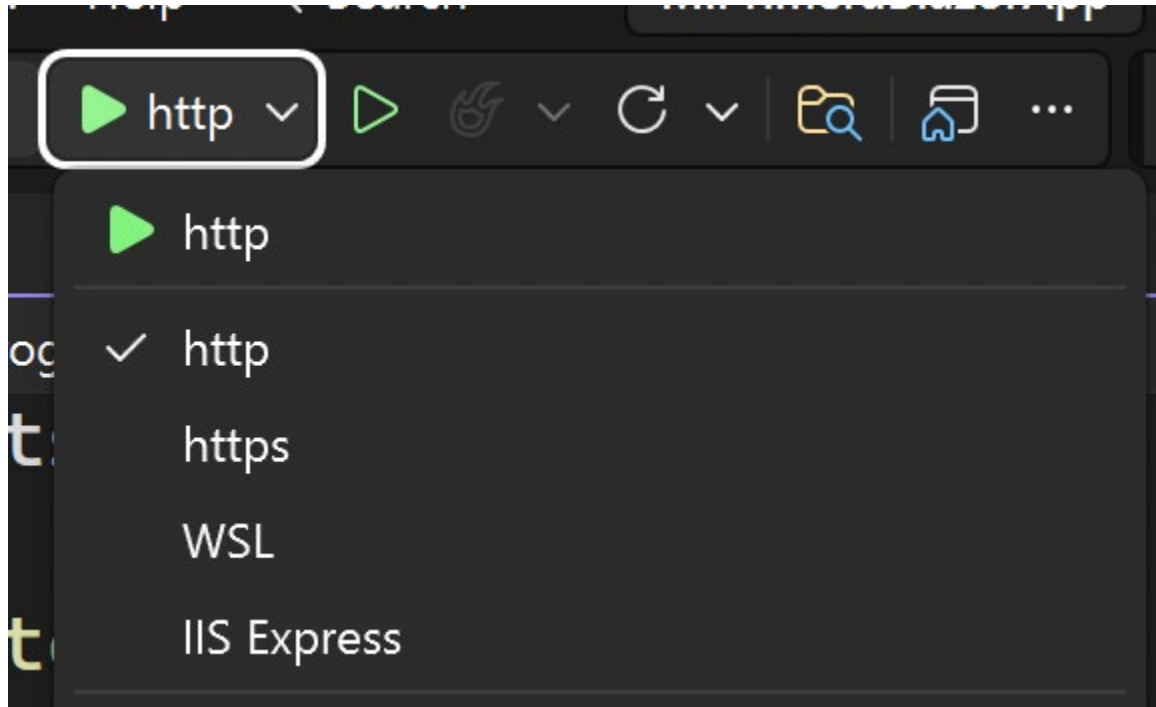
☐ Use the .dev.localhost TLD in the application URL ⓘ

☐ Enlist in Aspire orchestration ⓘ

Aspire version ⓘ

13.1

Iniciar la Aplicación Blazor



Sintaxis Básica de Razor

Sintaxis Razor Básica

Transición entre HTML y C#

- Razor es una sintaxis de marcado que permite incrustar código basado en servidor en páginas web. Se utiliza el carácter @ para realizar la transición del marcado HTML a C#.
- Las expresiones C# son evaluadas y el resultado se incrusta en el HTML renderizado.

Expresiones y Variables

```
Hola, @nombre  
La fecha actual es: @DateTime.Now  
Resultado: @(2 + 2)  
  
@code {  
    private string nombre = "Estudiante" ;  
}
```

Bloques de Código @code

Estructura Lógica

- El bloque @code permite añadir miembros de clase de C# a un componente Razor. Aquí se declaran campos, propiedades y métodos.
- El compilador genera una clase detrás de escena donde el HTML se convierte en el método de renderizado y el bloque @code provee los miembros de la clase.

Bloque de código

```
<button  
    @onclick="Incrementar" >Clic </button>  
Contador: @conteo  
  
@code {  
    private int conteo = 0;  
  
    private void Incrementar()  
    {  
        conteo++;  
    }  
}
```

Estructuras de Control en Razor

Renderizado Condicional e Iterativo

- Cualquier estructura de control de C# es válida en Razor añadiendo @ al inicio.
- Se utilizan frecuentemente sentencias if/else para condicionar la visibilidad de elementos, y foreach para generar listas de elementos HTML dinámicamente.

Uso de if y foreach

```
@if (elementos == null) {  
    Cargando datos ...  
} else {  
  
    @foreach (var item in elementos) {  
        @item.Nombre  
    }  
  
}
```

Router

El Componente Router

- El enrutamiento en Blazor se configura en el archivo `Components/Routes.razor`.
- El componente Router, generalmente referenciado en `App.razor`, analiza la URL del navegador. Busca un componente Razor que coincida con la ruta especificada.
- Si encuentra coincidencia, renderiza el componente.
- Si no hay coincidencia, procesa la ruta en la plantilla `NotFound`, mostrando un mensaje de error o redirigiendo. Esta plantilla se encuentra definida en `Components/Pages/NotFound.razor`

Creación de Rutas (@page)

La Directiva @page

- Para hacer que un componente sea accesible a través de una URL, se debe colocar la directiva @page en la primera línea del archivo .razor.
- Un solo componente puede tener múltiples directivas @page si se requiere que responda a múltiples URLs.

Implementación

```
@page "/inicio"  
@page "/home"
```

```
Página de Inicio  
Bienvenido a la aplicación.
```

Parámetros de Ruta

Paso de datos en la URL

- Las rutas pueden incluir parámetros que el componente utilizará. Los parámetros se definen entre llaves {} en la directiva @page.
- El componente debe tener una propiedad pública decorada con el atributo [Parameter] cuyo nombre coincida exactamente con el definido en la ruta.

Ejemplo de Parámetro

```
@page "/producto/{Id}"  
  
Detalle del Producto: @Id  
  
@code {  
    [Parameter]  
    public string Id { get; set; }  
}
```

Parámetros de Ruta Opcionales

Manejo de rutas flexibles

- Para definir que un parámetro no es obligatorio, se añade un signo de interrogación ? al final del nombre en la definición de la ruta.
- La propiedad receptora debe ser evaluada para comprobar si es nula o si contiene un valor por defecto.

Ejemplo de Parámetro Opcional

```
@page "/articulos/{Categoria?}"  
  
Viendo: @(Categoria ?? "Todo")  
  
@code {  
    [Parameter]  
    public string? Categoria { get; set; }  
}
```

Restricciones de Ruta

Tipado de URL

- Es posible restringir un parámetro de ruta a un tipo de dato específico, como enteros, decimales o identificadores únicos (GUID).
- Si la URL no coincide con la restricción de tipo, el enrutador considerará que no hay coincidencia y mostrará el estado de "No encontrado".

Implementación de Restricciones

```
@page "/usuario/{Id:int}"  
  
Usuario Número: @Id  
  
@code {  
    [Parameter]  
    public int Id { get; set; }  
}
```

Páginas, Componentes, Formularios y Modo Interactivo

Creación de Páginas

- Una página en Blazor es fundamentalmente un componente Razor con la directiva `@page`.
- Físicamente, es un archivo con extensión `.razor`.
- Convención: El nombre del archivo debe comenzar con letra mayúscula (por ejemplo, `Index.razor` o `ListaUsuarios.razor`).
- Se recomienda agrupar los componentes enrutables en la carpeta `Pages` o `Components/Pages` del proyecto.
- Toda página asume el diseño (Layout) predeterminado definido en `App.razor`, pero puede ser sobrescrito localmente con `@layout`.

Componentes

- Los componentes sin directiva @page actúan como bloques de construcción reutilizables.
- Pueden anidarse dentro de otros componentes o páginas.
- Permiten encapsular la lógica y la interfaz de una sección específica (por ejemplo, un menú lateral, una tarjeta de producto o un modal).
- Promueven el principio de responsabilidad única en la presentación de datos.

Definición e Invocación de Componentes

Creación del Componente
(Tarjeta.razor)

```
<div  
  class= "card">  
    Componente Reutilizable  
    Contenido interno de la tarjeta.  
</div>
```

Invocación desde otra página

```
@page "/vista"  
  
Vista Principal  
  
<Tarjeta />  
<Tarjeta />
```

Parámetros de Componentes ([Parameter])

Comunicación Padre a Hijo

- Para enviar datos a un componente hijo, se definen propiedades públicas en el hijo decoradas con [Parameter].
- El componente padre asigna valores a estos parámetros utilizando atributos HTML que coinciden con el nombre de la propiedad.

Ejemplo de Paso de Parámetros

```
<Saludo  
  NombreUsuario = "Juan" />  
  
Bienvenido, @NombreUsuario  
  
@code {  
  [Parameter]  
  public string NombreUsuario { get; set; }  
}
```

Eventos (EventCallbacks)

Comunicación Hijo a Padre

- Para que un componente hijo notifique a su componente padre sobre una acción, se utiliza el tipo EventCallback o EventCallback.
- El hijo invoca el callback y el padre asocia un método local que responderá a dicha invocación.

```
@* BotonAccion.razor *@
```

```
<button class="btn btn-primary" @onclick="NotificarPadre">  
    Haz clic para enviar datos  
</button>
```

```
@code {  
    [Parameter]  
    public EventCallback<string> OnAccion { get; set; }  
  
    private async Task NotificarPadre()  
    {  
        // Al ejecutar InvokeAsync, enviamos el string "Datos" al padre  
        await OnAccion.InvokeAsync("Datos desde el Hijo");  
    }  
}
```

```
@* PaginaPrincipal.razor *@
```

```
<h3>Mensaje recibido: @mensajeDelHijo</h3>
```

```
<BotonAccion OnAccion="ProcesarMensaje" />
```

```
@code {  
    private string mensajeDelHijo = "Esperando...";  
  
    private void ProcesarMensaje(string valorRecibido)  
    {  
        // Esta lógica se ejecuta cuando el hijo hace clic en el botón  
        mensajeDelHijo = valorRecibido;  
    }  
}
```

Ciclo de Vida del Componente

- Blazor expone métodos virtuales que pueden ser sobrescritos para ejecutar lógica en momentos precisos de la existencia del componente.
- `OnInitialized` / `OnInitializedAsync`: Se ejecuta cuando el componente ha sido inicializado y ha recibido sus parámetros iniciales. Ideal para peticiones HTTP iniciales.
- `OnParametersSet` / `OnParametersSetAsync`: Se ejecuta después de la inicialización y cada vez que el componente padre cambia los parámetros que suministra al hijo.
- `OnAfterRender` / `OnAfterRenderAsync`: Se ejecuta después de que el componente ha terminado de renderizarse en el DOM. Necesario para llamadas de interoperabilidad con JavaScript (JSInterop).

Implementación de Métodos del Ciclo de Vida

Sobrescritura de Métodos Base

- Al heredar de `ComponentBase`, la directiva `override` es obligatoria.
- Las versiones asíncronas de estos métodos son recomendadas cuando se efectúan lecturas de disco, base de datos o peticiones de red para no bloquear el hilo de ejecución principal.

Ejemplo de `OnInitializedAsync`

```
@code {  
    private List lista;  
  
    protected override async Task  
    OnInitializedAsync()  
    {  
        // Llamada a servicio asíncrono  
        lista = await  
        ServicioDB.ObtenerTodos();  
    }  
}
```

*** `OnInitializedAsync` devuelve `Task.CompletedTask`*

Manejo de Estado

Estrategias de Manejo de Estado

- Debido a que los componentes son instancias temporales, el estado se pierde en una recarga completa de la página.
- **Estado en memoria:** Se inyecta una clase compartida donde los componentes leen y escriben datos temporales.
- **Almacenamiento del Navegador:** Uso de LocalStorage y SessionStorage mediante bibliotecas o JavaScript Interop para persistir datos entre recargas.
- **Estado en URL:** Uso de parámetros de ruta y Cadenas de Consulta (Query Strings) para estados navegables y compartibles.

Manejo de Query Strings

Atributo SupplyParameterFromQuery

- Para recuperar valores directamente de la cadena de consulta de la URL (por ejemplo: /pagina?orden=asc), se utiliza el atributo [SupplyParameterFromQuery].
- Es indispensable para parámetros de búsqueda o filtrado en listas.

Implementación

```
@page "/buscar"

Buscando por: @Termino

@code {
    [SupplyParameterFromQuery (Name = "q")]
    public string Termino { get; set; }
}

// URL: /buscar?q=Blazor
```

Navegación con NavigationManager

Inyección de Dependencia

NavigationManager es un servicio del sistema cuya referencia se inyecta usando Inyección de Dependencias a través de @inject.

```
@inject NavigationManager  
Nav
```

Ejemplo de uso

```
@page "/productos"  
@using MiPrimeraBlazorApp.Components.Shared  
@using MiPrimeraBlazorApp.Modelo
```

```
@inject NavigationManager Nav
```

```
<h3>@Titulo</h3>
```

```
....
```

```
public void ModificarProducto(int codigo)  
{  
    Nav.NavigateTo($"producto/{codigo}");  
}
```

Formularios en Blazor

- Blazor ofrece un componente especializado llamado EditForm para gestionar entradas del usuario y validación de forma centralizada.
- El componente vincula un objeto del modelo de datos de C# a los campos de entrada visuales (Two-way Data Binding).
- Gestiona eventos nativos para envíos válidos o inválidos, facilitando la validación del modelo antes del procesamiento.

El Componente EditForm

Estructura del Formulario

Se requiere el parámetro Model asignado a la instancia del objeto de datos.

El evento OnValidSubmit asegura que la función especificada solo se ejecute si todas las reglas de validación han sido superadas.

```
@using System.ComponentModel.DataAnnotations

<EditForm Model="@usuario" OnValidSubmit="Guardar">
    @* Habilita las validaciones basadas en Data Annotations *@
    <DataAnnotationsValidator />
    <ValidationSummary />

    <div class="mb-3">
        <label>Nombre:</label>
        @* Usamos InputText para que bind-Value funcione correctamente *@
        <InputText @bind-Value="usuario.Nombre" class="form-control" />
        <ValidationMessage For="@() => usuario.Nombre" />
    </div>

    <button type="submit" class="btn btn-success">Enviar</button>
</EditForm>

@code {
    private Usuario usuario = new Usuario();

    private void Guardar()
    {
        // Esta lógica solo se ejecuta si el formulario es válido
        Console.WriteLine($"Usuario guardado: {usuario.Nombre}");
    }

    // Clase de modelo con reglas de validación
    public class Usuario
    {
        [Required(ErrorMessage = "El nombre es obligatorio")]
        [StringLength(20, ErrorMessage = "El nombre es demasiado largo")]
        public string Nombre { get; set; } = string.Empty;
    }
}
```

Componentes de Entrada Integrados

Blazor proporciona componentes integrados que heredan el estilo de comportamiento de validación de EditForm.

InputText: Se renderiza como una caja de texto.

InputNumber: Se renderiza como un caja de texto y parsea fuertemente el valor a tipos numéricos en C#.

InputDate: Se renderiza como un DatePicker para objetos DateTime.

InputCheckbox / InputSelect: Para valores booleanos y selecciones de lista desplegable.

Siempre utilizan la directiva @bind-Value para lograr el enlace bidireccional (Two-way Data Binding).

Modos Interactivos (Render Modes en .NET 8+)

Static Server

Modo por defecto. El componente emite HTML estándar que se envía al cliente sin conexión SignalR ni carga de WebAssembly. No permite botones interactivos (excepto envíos de formulario nativos).

Interactive Server

Establece conexión WebSocket. Maneja interactividad rápida en la red interna, delegando ejecución al servidor.
(Atributo: @rendermode InteractiveServer)

.

Interactive WebAssembly

Carga el runtime en el navegador y ejecuta lógica del lado cliente. Ideal para aplicaciones que no deben congestionar recursos del servidor.
(Atributo: @rendermode InteractiveWebAssembly).

Code Behind

Separación de Código e Interfaz

- El patrón Code Behind consiste en separar el marcado HTML de la lógica de programación en C#.
- Evita que archivos .razor complejos crezcan de forma excesiva e inmanejable en el bloque @code.
- Fomenta prácticas de código limpio y facilita la colaboración (diseñadores trabajando en HTML y programadores en C#).
- Blazor logra esta separación utilizando el concepto de clases parciales (partial classes) de C#.

Implementación de Code Behind

Reglas de Estructura

Si el componente se llama Vista.razor, se crea un archivo C# en el mismo directorio nombrado Vista.razor.cs.

La clase definida debe tener el mismo nombre del componente, estar dentro del mismo espacio de nombres y obligatoriamente declararse con el modificador partial.

Archivo Vista.razor.cs

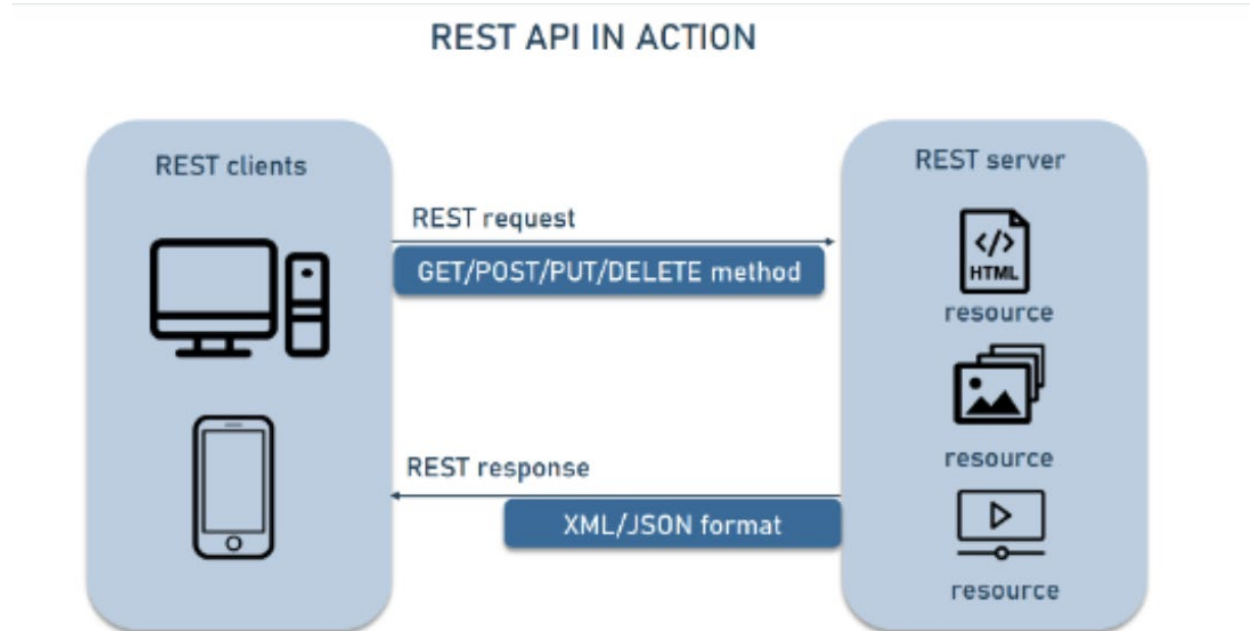
```
namespace MyApp.Pages
{
    public partial class Vista
    {
        private string Mensaje = "Cargado";

        protected override void
        OnInitialized()
        {
            // Lógica compleja aislada del HTML
        }
    }
}
```

Integración con un API REST

HttpClient en Blazor

- Para establecer comunicación con servicios externos o backends, Blazor utiliza la clase.
- HttpClient de .NET.
- El servicio HttpClient se registra globalmente en Program.cs y se inyecta en los componentes que lo requieran.
- Proporciona métodos asíncronos para los verbos HTTP estándar (GET, POST, PUT, DELETE).
- Existen métodos de extensión como GetFromJsonAsync que deserializan automáticamente la respuesta JSON en objetos C#.



Realizar Peticiones GET

Lectura de Datos

Se inyecta la instancia de `HttpClient` y se invoca la petición usualmente dentro del método **`OnInitializedAsync`**.

La operación asíncrona no bloqueará la interfaz y, una vez finalizada, Blazor refrescará automáticamente la vista.

Ejemplo de Solicitud GET

```
@inject HttpClient Http

@code {
    private List pronosticos;

    protected override async Task
    OnInitializedAsync()
    {
        pronosticos = await
            Http.GetFromJsonAsync
                >("api/clima" );
    }
}
```

Realizar Peticiones POST

Envío de Datos al Servidor

Las peticiones para crear o modificar recursos requieren adjuntar el objeto como cuerpo (body) de la solicitud.

El método `PostAsJsonAsync` serializa el objeto automáticamente e incluye las cabeceras correspondientes de `application/json`.

Ejemplo de Solicitud POST

```
@code {  
    private NuevoUsuario user = new();  
  
    private async Task Crear()  
    {  
        var resp = await  
Http.PostAsJsonAsync(  
        "api/usuarios", user);  
  
        if (resp.IsSuccessStatusCode)  
        {  
            // Navegar o notificar éxito  
        }  
    }  
}
```

Inyección de Dependencias

Registro e Inyección de Servicios

Concepto de DI en Blazor

- La Inyección de Dependencias (DI) permite crear aplicaciones débilmente acopladas. Los servicios se registran en Program.cs con un ciclo de vida específico: Transient, Scoped, o Singleton.
- En los componentes Razor, la inyección se materializa usando la directiva `@inject` en la cabecera del archivo.

Ciclo DI: Registro y Uso

```
// 1. En Program.cs
builder.Services.AddScoped();

// 2. En Componente.razor
@inject IMiServicio Servicio

@onclick="Ejecutar" >Acción
@code {
    void Ejecutar() =>
        Servicio.RealizarOperacion();
}
```

Validación de Entradas

DataAnnotations y ValidationSummary

Validación Declarativa

- Se decoran las propiedades del modelo con atributos del espacio de nombres `System.ComponentModel.DataAnnotations`.
- El componente `DataAnnotationsValidator` adjuntado a `EditForm` evaluará estas reglas. `ValidationSummary` y `ValidationMessage` mostrarán los mensajes de error en la interfaz.

Modelo con Anotaciones

```
public class UsuarioModel
{
    [Required(ErrorMessage = "El nombre es obligatorio")]
    public string Nombre { get; set; }

    [EmailAddress]
    public string Correo { get; set; }
}
```

Localización

Localización de la Aplicación

```
string[] supportedCultures = ["es-PE"];  
var localizationOptions = new RequestLocalizationOptions()  
    .SetDefaultCulture(supportedCultures[0])  
    .AddSupportedCultures(supportedCultures)  
    .AddSupportedUICultures(supportedCultures);  
  
app.UseRequestLocalization(localizationOptions);
```

Autenticación y Autorización

Conceptos de Seguridad en Blazor

- Blazor se apoya en los mecanismos de identidad de ASP.NET Core subyacentes.
- El servicio principal es el proveedor de estado de autenticación.
- El proveedor notifica al entorno de renderizado cuando la identidad del usuario activo cambia.
- La lógica de autenticación (obtención de tokens o cookies) depende de si se usa el modelo WebAssembly o Server.

Habilitar Autenticación y Autorización para Server Mode

```
var builder = WebApplication.CreateBuilder(args);  
...  
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)  
    .AddCookie(opciones => {  
        opciones.LoginPath = "/Login";  
        opciones.ExpireTimeSpan = TimeSpan.FromHours(8);  
    });  
builder.Services.AddAuthorization();  
builder.Services.AddCascadingAuthenticationState();  
var app = builder.Build();  
...  
app.UseAuthentication();  
app.UseAuthorization();  
...  
app.Run();
```

Injectar un Servicio de Autenticación

Program.cs

```
builder.Services.AddHttpContextAccessor();  
builder.Services.AddScoped<IServicioSesionAuth,  
ServicioSesionAuth>();
```

```
app.MapAuthEndpoints();
```

Métodos de Extensión para `app.MapAuthEndpoints();`

```
public static class AuthEndpoints {  
    public static IEndpointRouteBuilder MapAuthEndpoints(this  
        IEndpointRouteBuilder endpoints) {  
        endpoints.MapPost("/auth/login", (HttpContext context,  
            IServicioSesionAuth servicioSesionAuth) =>  
            servicioSesionAuth.IniciarSesionAsync(context));  
  
        endpoints.MapGet("/auth/logout", (HttpContext context,  
            IServicioSesionAuth servicioSesionAuth) =>  
            servicioSesionAuth.CerrarSesionAsync(context));  
  
        return endpoints;  
    }  
}
```

Configurar el Router para Autenticación - Routes.razor

```
@rendermode InteractiveServer
```

```
<CascadingAuthenticationState>  
  <Router AppAssembly="typeof(Program).Assembly" NotFoundPage="typeof(Pages.NotFound)">  
    <Found Context="routeData">  
      <AuthorizeRouteView RouteData="routeData" DefaultLayout="typeof(Layout.MainLayout)">  
        <NotAuthorized>  
          <RedirectToLogin />  
        </NotAuthorized>  
      </AuthorizeRouteView>  
      <FocusOnNavigate RouteData="routeData" Selector="h1" />  
    </Found>  
  </Router>  
</CascadingAuthenticationState>
```

Componente AuthorizeView

El componente AuthorizeView controla qué elementos del DOM se renderizan basado en los roles o el estado de inicio de sesión del usuario.

Posee plantillas anidadas (Authorized y NotAuthorized) para manejar de forma limpia ambas condiciones sin necesidad de bucles lógicos C#.

```
<AuthorizeView Roles="Admin">
  <Authorized>
    @* Este contenido solo se muestra si el usuario tiene el rol "Admin" *@
    <h3>Panel de Administración</h3>
    <p>Bienvenido, administrador. Aquí tienes acceso a los reportes.</p>
    <button class="btn btn-danger">Eliminar Usuarios</button>
  </Authorized>

  <NotAuthorized>
    @* Este contenido se muestra si el usuario NO está autenticado
    o si está autenticado pero NO tiene el rol "Admin" *@
    <div class="alert alert-warning">
      No posees permisos para ver esto. Por favor, contacta al soporte.
    </div>
  </NotAuthorized>
</AuthorizeView>
```

Gracias

